

CDCP - Classificador de Chamados Públicos

JUSTIFICATIVA DE TECNOLOGIAS UTILIZADAS

1. Justificativa da mensageria

1.1 Problema que a mensageria resolve

A abertura de um chamado envolve uma operação simples para o cidadão, mas desencadeia tarefas de processamento que podem exigir mais tempo:

Sem mensageria, a API poderia precisar aguardar o processamento do PLN e da classificação antes de finalizar a requisição do cidadão.

Isso criaria acoplamento entre API, PLN, modelo de classificação, banco de dados e tempo de resposta da aplicação.

Com RabbitMQ, a API registra o chamado e publica uma mensagem para processamento posterior.

A documentação do projeto define exatamente esse fluxo: primeiro são cadastrados os dados-base do chamado e, depois, a mensageria aciona os processos de PLN e IA, executados paralelamente.

1.2 Desacoplamento

O principal benefício é separar a API dos consumidores responsáveis pelo processamento.

A API não precisa conhecer todos os detalhes de execução dos modelos. Ela apenas publica uma mensagem contendo a referência do chamado e os dados necessários para o processamento.

Isso facilita manutenção, evolução dos modelos, substituição de consumidores, execução independente, tratamento de falhas e escalabilidade futura.

1.3 Processamento assíncrono

A abertura do chamado não precisa ficar bloqueada enquanto o PLN e o CART trabalham.

Isso é importante porque o fluxo funcional estabelece um estado de **Processamento** antes de o chamado chegar ao estado **Pendente**. Depois da classificação, o chamado passa para Pendente e pode ser analisado pelo gestor.

1.4 Execução independente dos consumidores

PLN e CART podem ser tratados como consumidores independentes.

Essa separação evita que uma etapa precise necessariamente executar dentro da mesma requisição HTTP.

2. RabbitMQ em ambiente de nuvem

O projeto prevê o RabbitMQ hospedado na mesma máquina virtual em que ficarão os demais serviços da aplicação.

Para o contexto acadêmico e para o porte inicial da aplicação, essa decisão é justificável porque reduz custo, complexidade operacional, quantidade de serviços externos e dificuldade de configuração.

É uma escolha adequada para o cenário inicial do projeto. Caso o volume de chamados aumente significativamente, a arquitetura pode evoluir para RabbitMQ em infraestrutura dedicada, múltiplos consumidores, filas separadas por tipo de processamento, escalonamento horizontal, banco gerenciado e serviços de PLN/IA independentes.

Portanto, a solução atual privilegia **simplicidade e custo**, mantendo uma arquitetura que pode evoluir posteriormente.

3. Confiabilidade proporcionada pela mensageria

O RabbitMQ cria uma camada intermediária entre quem produz uma tarefa e quem a executa.

Isso permite trabalhar com conceitos como:

- Filas;
- Consumidores;
- Confirmação de processamento;
- Reentrega de mensagens;
- Mensagens persistentes;
- Controle de concorrência.

No projeto, isso é útil porque um erro temporário no PLN ou no classificador não deveria fazer o cidadão perder o chamado já criado.

O registro do chamado deve existir independentemente da conclusão do processamento inteligente.

4. Justificativa do CART

4.1 Objetivo da classificação

O modelo possui uma função operacional clara: **determinar a urgência de um chamado**.

As classes utilizadas são: BAIXA, MÉDIA, ALTA e URGENTE.

O modelo recebe informações estruturadas do chamado, incluindo dados identificados pelo PLN.

A documentação estabelece que o PLN deve identificar informações como tipo do chamado, risco à vida humana, risco à vida animal e bloqueio de via, e que essas informações serão utilizadas na classificação da urgência.

5. Características utilizadas pelo modelo

A documentação de treinamento informa que a base possui os seguintes atributos:

Atributo	Tipo	Utilização
ID	Identificador	Removido
Data de criação	Data	Removida
Tipo do chamado	Categórico	Mantido
Dias do problema	Numérico	Mantido
Risco à vida humana	Binário	Mantido
Risco à vida animal	Binário	Mantido
Bloqueio de via	Binário	Mantido
Urgência	Classe	Variável-alvo

O ID foi removido por não possuir valor analítico, enquanto a data de criação também foi retirada porque o chamado deve ser classificado considerando as informações do problema no momento da abertura.

6. Preparação da base de treinamento

6.1 Remoção de atributos irrelevantes

Foram removidos ID e data de criação; foram mantidos os atributos relacionados diretamente às características do problema.

6.2 Dados ausentes

A documentação registra que não foram encontrados atributos ausentes ou nulos.

6.3 Dados inconsistentes/redundantes

Foram removidas 3.228 linhas de uma base original de 8.154 registros.

6.4 Normalização

O atributo numérico **dias_problema** foi normalizado, levando a média para aproximadamente zero e o desvio padrão para um.

6.5 Outliers

A documentação registra que não foram encontrados outliers em **dias_problema**.

7. Distribuição das classes

Após o processamento da base, a documentação registra:

Classe	Quantidade
URGENTE	3.484
ALTA	741
MÉDIA	579
BAIXA	122
Total	4.926

Esse resultado mostra um problema importante de **desbalanceamento das classes**: a classe URGENTE possui quantidade muito superior à classe BAIXA.

Esse aspecto deve ser considerado na avaliação do modelo, pois uma acurácia elevada pode esconder desempenho ruim nas classes menos representadas.

8. Técnica de treinamento

A estratégia documentada foi a separação da base em dois conjuntos: Conjunto de Treinamento e Conjunto de Validação.

Essa estratégia corresponde a uma abordagem de **holdout**, na qual parte dos dados é utilizada para ajustar o modelo e outra parte é reservada para avaliar seu desempenho em dados não utilizados no treinamento.

9. Comparação dos algoritmos

Antes da escolha do modelo definitivo, foram testados os modelos: LR, LDA, KNN, **CART**, NB e SVM.

Os resultados documentados foram:

Algoritmo	Precisão
LR	78,48%
LDA	78,70%
KNN	81,72%
CART	82,57%
NB	75,80%
SVM	79,85%

O CART apresentou o melhor resultado entre os modelos testados. Por isso, sua escolha foi baseada em comparação experimental, e não apenas em preferência tecnológica.

10. Por que CART é adequado ao projeto

10.1 Interpretabilidade

Uma árvore de decisão é relativamente fácil de interpretar.

Isso é interessante em um sistema público porque o resultado precisa ser compreensível para a equipe responsável pelo sistema.

10.2 Adequação aos atributos

O problema possui atributos binários, categóricos e numéricos. Árvores de decisão são adequadas para representar decisões baseadas em diferentes atributos e pontos de separação.

10.3 Baixo custo de inferência

Depois de treinado, o modelo realiza a classificação percorrendo os nós da árvore até alcançar uma folha, o que é adequado para processar muitos chamados.

11. Técnica de mineração de dados — Clustering

11.1 Objetivo

A classificação responde:

Qual é a urgência deste chamado?

O agrupamento responderá:

Quais chamados possuem características semelhantes?

A classificação possui uma variável-alvo conhecida (urgência); o clustering não possui classe previamente definida e busca descobrir estruturas presentes nos dados.

12. Por que utilizar clustering

O histórico de chamados pode conter padrões que não foram explicitamente definidos nas regras de negócio.

Exemplo:

Cluster A

- muitos dias de problema;
- risco à vida humana;
- bloqueio de via;
- determinados tipos de chamado.

Outro grupo pode apresentar:

Cluster B

- poucos dias de problema;
- sem risco;
- sem bloqueio;
- determinados tipos de manutenção.

Esses grupos podem posteriormente ser apresentados no dashboard para apoiar a análise dos gestores.

13. Variáveis do clustering

A primeira proposta é utilizar:

Variável	Tipo
Tipo do chamado	Categórica
Dias do problema	Numérica
Risco à vida humana	Binária
Risco à vida animal	Binária
Bloqueio de via	Binária

14. Resultado esperado do clustering

A aplicação deverá interpretar as características predominantes de cada grupo.

Exemplo:

Cluster 1 — Ocorrências críticas

Quantidade: 184 chamados

Características predominantes:

- bloqueio de via;
- presença de risco humano;
- determinados tipos de ocorrência.

15. Integração do clustering com a arquitetura

O clustering não precisa ser executado na abertura de cada chamado.

A classificação de urgência possui caráter operacional e precisa ocorrer no fluxo do chamado. Já a mineração por agrupamento possui caráter analítico e pode ser executada periodicamente.

Uma execução diária, semanal ou sob demanda pode ser suficiente para o primeiro estágio do projeto.

16. Benefícios esperados

Mensageria

- Desacoplamento;
- Processamento assíncrono;
- Melhor experiência na abertura do chamado;
- Possibilidade de processamento paralelo;
- Tolerância a falhas;
- Escalabilidade futura.

CART

- Melhor resultado entre os algoritmos testados;
- Interpretação relativamente simples;
- Baixo custo de inferência;
- Integração com os dados extraídos pelo PLN;
- Classificação automática da urgência.

Clustering

- Descoberta de padrões;

- Identificação de perfis de chamados;
- Apoio à tomada de decisão;
- Análise do histórico;
- Possibilidade de descobrir problemas recorrentes;
- Complementação da classificação supervisionada.

16. Conclusão

A arquitetura proposta combina três mecanismos com responsabilidades diferentes.

O **RabbitMQ** é responsável pela comunicação assíncrona entre a API e os serviços de processamento, evitando que a abertura de um chamado fique diretamente dependente da execução do PLN e do classificador.

O **CART** é utilizado na classificação supervisionada da urgência porque apresentou o melhor resultado entre os algoritmos avaliados, com 82,57% de precisão no experimento documentado.

O processo de **Cluster** será utilizado como técnica de mineração de dados para descobrir agrupamentos no histórico de chamados, funcionando de maneira complementar ao CART. Enquanto o CART responde qual é a urgência de um chamado, o clustering procura identificar padrões de comportamento e características semelhantes entre conjuntos de chamados.